

Intelheath OpenMRS Setup for FHIR2 v2.0 Compatibility

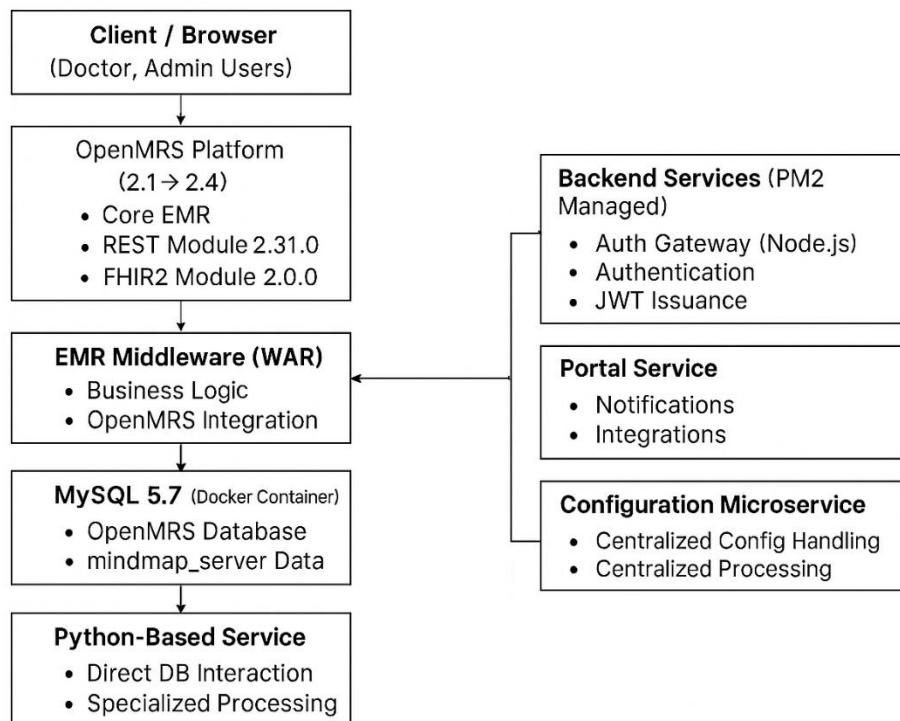
1 Introduction

This document is intended to describe the technical steps required for setting up the Intelheath OpenMRS environment. The setup is required to enable compatibility between Intelheath OpenMRS modules and FHIR2 version 2.0. FHIR2 is implemented as an OpenMRS module, and compatibility requirements necessitate the preparation of a compliant OpenMRS runtime environment. The procedures documented in this specification reflect the exact steps executed during environment preparation and are designed to allow accurate reproduction of the setup process.

2 Scope

This document covers system preparation, database configuration using Dockerized MySQL 5.7, Apache Tomcat installation, OpenMRS Platform and Reference Application deployment, REST module installation, and initial setup steps for the Intelheath Middleware.

3 System Architecture Overview



4 System Preparation

The base system was prepared to support Java-based and containerized services required for OpenMRS and associated applications.

What is done	Why this is required	Expected result
The Java is installed.	OpenMRS is a Java-based application. Without the correct Java version, OpenMRS and related services cannot run. Updating system packages avoids dependency conflicts later.	The system is ready to run Java applications and containerized services.

4.1 Update System Packages

System package metadata was updated to ensure access to the latest repository information.

Command
<code>sudo apt-get update</code>

4.2 Install OpenJDK 8

OpenJDK 8 was installed as the Java runtime required by OpenMRS.

Command
<code>sudo apt-get install openjdk-8-jdk</code>

5 Database Setup (MySQL 5.7 Using Docker)

MySQL 5.7 was required for compatibility. To avoid direct host installation, MySQL was deployed using Docker.

What is done	Why this is required	Expected result
A MySQL 5.7 database is deployed using Docker, and an existing OpenMRS database is restored.	OpenMRS depends on MySQL for data storage. MySQL 5.7 is used to maintain compatibility with existing schemas and modules. Docker is used to isolate the database and simplify management.	A running MySQL service that contains valid OpenMRS data and is accessible to applications.

5.1 Create MySQL Working Directory

Command
<code>mkdir ~/mysql-docker</code> <code>cd ~/mysql-docker</code> <code>mkdir config</code>

5.2 Configure MySQL Server Parameters

A custom MySQL configuration file was created.

Command
nano config/my.cnf

Configuration
[mysqld] innodb_buffer_pool_size = 1G innodb_log_file_size = 256M innodb_flush_log_at_trx_commit = 1 innodb_file_per_table = 1 query_cache_type = 1 query_cache_size = 64M max_allowed_packet=64M bind-address = 0.0.0.0 max_connections = 200 wait_timeout = 3600 interactive_timeout = 3600 slow_query_log = 1 slow_query_log_file = /var/lib/mysql/mysql-slow.log long_query_time = 2 skip-symbolic-links = 1 [client] max_allowed_packet=64M

5.3 Configure Docker Compose for MySQL

Command
nano docker-compose.yml

Docker Compose Configuration

```
services:
  mysql:
    image: fahadkabir/mysql:5.7
    container_name: mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: openmrs
      MYSQL_DATABASE: openmrs
      MYSQL_USER: openmrs
      MYSQL_PASSWORD: openmrs
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
      - ./config:/etc/mysql/conf.d
volumes:
  db_data:
```

5.4 Restore Existing Database

An existing OpenMRS database backup was restored into the Dockerized MySQL instance.

6 Apache Tomcat Installation

Apache Tomcat was installed as the application server.

What is done	Why this is required	Expected result
Apache Tomcat is installed and configured.	Tomcat is the servlet container that runs OpenMRS, the middleware WAR files and the web applications.	A stable Java application server ready to host OpenMRS and other services.

6.1 Download and Install Tomcat 8.5.100

Command

```
wget https://archive.apache.org/dist/tomcat/tomcat-8/v8.5.100/bin/apache-tomcat-8.5.100.zip
unzip apache-tomcat-8.5.100.zip
mv apache-tomcat-8.5.100 /opt/tomcat8
```

7 OpenMRS Core and Reference Application Setup

What is done	Why this is required	Expected result
OpenMRS Platform and the Reference Application modules are deployed. The REST module is installed for API access.	• OpenMRS Platform provides the core EMR functionality • The Reference Application enables clinical workflows • The REST module allows external applications and services to interact with OpenMRS	An operational OpenMRS instance accessible through a web browser.

7.1 Deploy OpenMRS Platform 2.1.4

Command
wget https://sourceforge.net/projects/openmrs/files/releases/OpenMRS_Platform_2.1.4/openmrs.war/download mv download /opt/tomcat8/webapps/openmrs.war

7.2 Deploy Reference Application 2.8.0

Command
wget https://sourceforge.net/projects/openmrs/files/releases/OpenMRS_Reference_Application_2.8.0/referenceapplication-addons-2.8.0.zip/download mv download refapp-2.8.0.zip unzip refapp-2.8.0.zip mv refapp-2.8.0 ~/.OpenMRS/modules/

7.3 Install REST Module 2.31.0

Command
wget https://addons.openmrs.org/api/v1/addon/org.openmrs.module.webservices-rest/2.31.0/download rm ~/.OpenMRS/modules/webservices.rest-*.omod mv webservices.rest-2.31.0.omod ~/.OpenMRS/modules/

7.4 Start Tomcat

Command
cd /opt/tomcat8/bin ./startup.sh

8 Intelhealth Middleware Setup

What is done	Why this is required	Expected result
The Intelhealth middleware is built using Maven and deployed to Tomcat.	The middleware acts as an integration layer between OpenMRS, external services, and frontend applications. It handles data processing, communication and business logic.	The middleware is deployed as a WAR file and successfully connects to OpenMRS.

8.1 Install Apache Maven 3.6.3

Command
wget https://archive.apache.org/dist/maven/maven-3/3.6.3/binaries/apache-maven-3.6.3-bin.zip unzip apache-maven-3.6.3-bin.zip mv apache-maven-3.6.3 /opt/maven/

8.2 Configure Maven Environment

Command
nano ~/.bashrc
Environment Variables
export M2_HOME=/opt/maven export PATH=\$PATH:\$M2_HOME/bin
Command
source ~/.bashrc

8.3 Build and Deploy Middleware

Command
git clone git@github.com:Intelhealth/intelehealth-middleware.git cd intelhealth-middleware/EMR-Middleware mvn clean package -Dmaven.test.skip=true cp target/EMR-Middleware.war /opt/tomcat8/webapps/

8.4 Restart Tomcat

Command
cd /opt/tomcat8/bin ./shutdown.sh ./startup.sh

9 Doctor Web Application Setup

What is done	Why this is required	Expected result
The doctor web application is built using Node.js and deployed to Tomcat.	This application provides the user interface used by clinicians to interact with patient data, visits, and workflows.	The web application is accessible through the browser and communicates with OpenMRS and backend services.

9.1 Install Node.js Using NVM

Command
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh bash

Terminal was restarted.

Command
nvm install 24.12.0 nvm use 24.12.0 nvm alias default 24.12.0

9.2 Build Doctor Web Application

Command
git clone git@github.com:Intelehealth/intelehealth-doctor-webapp.git cd intelehealth-doctor-webapp npm install npm run build

Command
cp -r dist/intelehealth-ui /opt/tomcat8/webapps/root

10 Backend Services Setup (PM2)

All backend services were installed and managed using PM2.

What is done	Why this is required	Expected result
--------------	----------------------	-----------------

Multiple backend services are configured and managed using PM2, including: • Authentication gateway • Portal service • WebRTC service • Configuration microservice	These services support authentication, real-time communication, notifications, and external configuration management.	All background services are running continuously and restart automatically if stopped.
---	---	--

10.1 Install PM2

Command
<code>npm install -g pm2@latest</code>

10.2 Deploy Auth Gateway, Portal, WebRTC, and Configuration Services

Each service was configured by creating a .env file, installing dependencies, and starting the service using PM2.

Command
<pre>npm install pm2 start bin/www --name pm2 save</pre>

11 Python Application Setup

What is done	Why this is required	Expected result
A Python-based service is configured and connected to the OpenMRS database.	This service performs specific processing tasks that are not handled by Java or Node.js components.	The Python service runs successfully and can access required database data.

11.1 Install Python Environment

Command
<pre>sudo apt-get install python3 python3-dev python3-venv python3 -m venv venv source venv/bin/activate pip install -r requirements.txt</pre>

11.2 Start Python Service

Command
python index.py

12 OpenMRS and Reference Application Upgrade Path

OpenMRS and Reference Application versions were upgraded sequentially to maintain compatibility.

What is done	Why this is required	Expected result
OpenMRS and the Reference Application are upgraded incrementally across multiple versions.	Direct upgrades across large version gaps can lead to database errors, incompatible modules, and startup failures. Incremental upgrades ensure: • Safe database migrations • Controlled module compatibility • Easier troubleshooting	The system remains stable after each upgrade step.

Each upgrade followed the same pattern:

- Stop Tomcat
- Replace WAR or modules
- Update configurations if required
- Start Tomcat

Command
cd /opt/tomcat8/bin sudo ./shutdown.sh sudo ./startup.sh

Special handling was required for outdated modules and FHIR2 conflicts, including:

- Removing incompatible FHIR2 modules
- Updating liquibase checksum entries
- Reinstalling compatible Reference Application modules

13 Final FHIR2 Upgrade

FHIR2 was upgraded through the OpenMRS UI to version **2.0.0**, followed by a Tomcat restart if required.

What is done	Why this is required	Expected result
--------------	----------------------	-----------------

The FHIR2 module is upgraded to version 2.0.0 after all platform upgrades are complete.	FHIR2 v2.0 depends on newer OpenMRS platform features and database structures. Upgrading it at the end avoids compatibility failures.	The OpenMRS system runs successfully with FHIR2 version 2.0 enabled.
---	---	---

14 OpenMRS and Reference Application Upgrade Timeline

This section describes the **step-by-step upgrade path** followed for OpenMRS and the Reference Application.

The upgrades were executed **incrementally**, one version at a time, to ensure database integrity, module compatibility, and system stability.

Direct upgrades across major versions were intentionally avoided.

General Upgrade Principles

Before starting the detailed steps, the following rules were followed for **every upgrade**:

- Apache Tomcat was stopped before replacing any WAR file or modules
- Database backups were taken before major version changes
- Only compatible modules were retained after each upgrade
- Tomcat was started only after verifying file replacement
- The system was validated after each upgrade before proceeding

These principles allowed safe progression through multiple OpenMRS and Reference Application versions.

Stage 1: Baseline System Version

Starting Point

- **OpenMRS Platform:** 2.1.4
- **Reference Application:** 2.8.0
- **REST Module:** 2.31.0

This baseline version was used to establish a stable starting environment before initiating upgrades.

Stage 2: Reference Application Upgrade to 2.8.1

What Was Upgraded

- Reference Application modules only

Why This Step Was Required

Minor Reference Application upgrades include:

- Module compatibility fixes
- UI improvements
- Bug fixes required for later platform upgrades

Process

1. Tomcat was stopped

2. Existing modules were reviewed
3. All modules **except** webservices.rest-2.31.0.omod were removed
4. Modules from Reference Application **2.8.1** were copied
5. Tomcat was started

Expected Outcome

- OpenMRS runs successfully with updated Reference Application modules

Stage 3: OpenMRS Platform Upgrade to 2.2

What Was Upgraded

- OpenMRS core (openmrs.war) from version **2.1.4** → **2.2**

Why This Step Was Required

This platform upgrade introduced:

- Database schema changes
- Platform-level improvements required for later releases

Process

1. Tomcat was stopped
2. Database and WAR file backups were taken
3. The existing openmrs.war file was replaced with version 2.2
4. Database connection URL was updated in openmrs-runtime.properties
5. Tomcat was started

Expected Outcome

- OpenMRS starts successfully on version 2.2
- Database updates complete without errors

Stage 4: Reference Application Upgrade to 2.9.0

What Was Upgraded

- Reference Application modules from **2.8.x** → **2.9.0**

Why This Step Was Required

Reference Application 2.9.0 is aligned with OpenMRS 2.2 and includes:

- Updated modules
- Compatibility fixes

Process

1. Tomcat was stopped
2. All modules except REST were removed
3. Modules from Reference Application 2.9.0 were copied
4. Tomcat was started

Expected Outcome

- OpenMRS loads with Reference Application 2.9.0 modules

Stage 5: OpenMRS Platform Upgrade to 2.3

What Was Upgraded

- OpenMRS Platform from **2.2 → 2.3**

Why This Step Was Required

This was a major platform update introducing:

- New framework components
- Additional database changes

Observed Issue

- Tomcat failed to start due to **incompatible older Reference Application modules**

Resolution

- Reference Application modules were upgraded further

Stage 6: Reference Application Upgrade to 2.10

What Was Upgraded

- Reference Application modules to **version 2.10**

Why This Step Was Required

Reference Application 2.10 is compatible with OpenMRS 2.3 and resolves startup issues caused by outdated modules.

Process

1. Tomcat was stopped
2. Old modules (except REST) were removed
3. Modules from Reference Application 2.10 were copied
4. Tomcat was started

Expected Outcome

- OpenMRS 2.3 starts successfully

Stage 7: OpenMRS Platform Upgrade to 2.3.1 → 2.3.2 → 2.3.3

What Was Upgraded

- OpenMRS core upgraded sequentially:
 - 2.3.1
 - 2.3.2
 - 2.3.3

Why These Steps Were Required

Patch versions provide:

- Security fixes
- Bug fixes
- Stability improvements

Process (Repeated for Each Version)

1. Tomcat was stopped
2. WAR file was replaced
3. Tomcat was started
4. Database updates completed automatically

Stage 8: FHIR2 Module Conflict Resolution (2.3.3)

Observed Issue

- OpenMRS failed to start due to **FHIR2 module incompatibility**

Resolution

- Existing FHIR2 module was removed
- Tomcat was restarted

Expected Outcome

- OpenMRS 2.3.3 starts successfully without FHIR2

Stage 9: OpenMRS Platform Upgrade to 2.4

What Was Upgraded

- OpenMRS core from **2.3.3** → **2.4**

Why This Step Was Required

OpenMRS 2.4 is required as a base platform for:

- Newer Reference Applications
- FHIR2 version 2.0.0 compatibility

Process

- Same controlled WAR replacement process was followed

Stage 10: Reference Application Upgrade to 2.12.0

Observed Issue

- Liquibase checksum mismatch related to FHIR2 database changes

Resolution

- Tomcat was stopped
- Liquibase checksum values were corrected directly in the database
- Tomcat was restarted

Expected Outcome

- Database migrations completed successfully
- OpenMRS starts without checksum errors

Stage 11: OpenMRS Platform Upgrade to 2.4.2 → 2.4.3

What Was Upgraded

- Final patch upgrades:
 - 2.4.2
 - 2.4.3

Why This Step Was Required

These versions provide:

- Final bug fixes
- Stabilized platform required for FHIR2 v2.0.0

Stage 12: Final FHIR2 Module Upgrade to 2.0.0

What Was Upgraded

- FHIR2 module upgraded via OpenMRS UI

Why This Step Was Done Last

FHIR2 version 2.0.0 depends on:

- OpenMRS 2.4.x
- Fully upgraded Reference Application modules

Performing this upgrade earlier caused startup failures.

Expected Outcome

- OpenMRS starts cleanly
- FHIR2 v2.0.0 is active and functional

Final State Summary

- **OpenMRS Platform:** 2.4.3
- **Reference Application:** 2.12.0
- **FHIR2 Module:** 2.0.0

The system is fully upgraded, stable, and ready for FHIR2-based integration.